

Test report

Milestone 3 evidence: *"Public GitHub repo(s) ... with **CI logs, coverage report** ..." and "API docs + **test reports with coverage/outcomes**".*

The CI run

[Run 36004503722](#), 2026-09-24 13:16 UTC.

GitHub deletes Actions logs after 90 days, so a **full copy lives in the repository**: [ci-run-36004503722.log](#). It is the raw output of `gh run view 36004503722 --log`, with color codes stripped. The only secrets that appear in it are CI test values (`JWT_SECRET: ci-secret-jamas-en-produccion`, "never in production").

Job	What it runs	Result
App TS	lint (Biome) · typecheck · test ID traceability · tests + coverage on all four TypeScript parts (api, web, shared, cardano) · dependency scan · build · smoke test of the compiled start command	✓
Aiken contracts	<code>aiken fmt --check</code> · <code>aiken check</code> · <code>aiken build</code> , and that the committed <code>plutus.json</code> is up to date	✓
Static analysis	Semgrep, rulesets <code>p/security-audit</code> + <code>p/owasp-top-ten</code>	✓ 128 rules, 536 files, 0 findings
E2E Playwright	the whole app against a migrated and seeded database, on mobile (390 px) and desktop (1440 px) — non-blocking job (<code>continue-on-error</code>)	⚠️ flaky, see below

The pipeline since 2026-10-07

The run above is still the source of this report's coverage numbers. Since 2026-10-07 CI has a different shape: **one job per concern, all in parallel, and all blocking except accessibility**. Render deploys only if every check on the commit passes (`autoDeployTrigger: checksPass`, [runbook](#) §2). The first full run with that shape is [Run 37569273172](#) (`22c2975`, 2026-10-07 03:59 UTC), copied in [ci-run-37569273172.log](#):

Job	What it runs	Time	Result
Lint + test IDs	Biome · test ID traceability	0:24	✓
Typecheck	<code>tsc</code> across the workspace	0:36	✓
Tests + coverage (web)	vitest with threshold	1:31	✓ 1774/1774 · 100% statements
Tests + coverage (api)	vitest with threshold	1:16	✓ 792 passed · 3 skipped · 100% statements
Tests + coverage (shared + cardano)	vitest with threshold	0:57	✓ 241/241 and 148/148 · 100% statements

Job	What it runs	Time	Result
Build + compiled start	<code>pnpm build</code> and Render's real <code>startCommand</code>	0:46	✅
Dependency scan	<code>pnpm audit</code> : full report and critical gate	0:15	✅ 0 critical (the report lists 6 low, 21 moderate, 10 high)
E2E Playwright, 4 shards	the whole app, mobile and desktop, each shard with its own seeded database	2:35 · 1:53 · 2:08 · 1:33	✅ 90/90 (29 · 18 · 27 · 16)
E2E accessibility	axe in a real browser; non-blocking by decision (SPEC-112)	2:26	✅ 10/10
Aiken contracts	<code>fmt --check · check · build · plutus.json</code> up to date	0:18	✅ 102/102
Static analysis	Semgrep, <code>p/security-audit</code> + <code>p/owasp-top-ten</code>	0:37	✅ 322 rules, 629 files, 0 findings

The whole run took **2:40**; with the single job it replaced, 5:16.

Results per package

Package	What it covers	Tests	Result
<code>contracts/</code> (Aiken)	the on-chain validator: stage state machine, thread token, evidence rules	102 (100 unit + 2 property)	✅ 102/102
<code>apps/api</code>	the API: routes, authorization, anchoring, state machine, migrations	712	✅ 709 passed · 3 skipped
<code>packages/cardano</code>	the anchoring port (simulated and real)	115	✅ 115/115
<code>packages/shared</code>	the shared Zod contract, the state machine, Merkle trees	210	✅ 210/210
<code>apps/web</code> (unit)	components, <code>i18n</code> , <code>ApiPort</code> checked against the OpenAPI document	1,593	✅ 1,592 passed · 1 expected <code>it.fails</code> (see below)
<code>apps/web</code> (E2E)	the flows of the four roles plus admin, on two viewports	90	⚠️ flaky, see below
Total (excl. E2E)		2,732	2,728 passed · 3 skipped · 1 expected fail · 0 failed

The 3 skipped tests are intentional: they are `apps/api/test/storage-s3.test.ts`, which runs against a real S3 (MinIO). The same applies to `packages/cardano/src/yaci.test.ts`, which runs against a local Cardano node and is excluded by that package's test script. CI does not start that infrastructure; both are run by hand with `pnpm --filter @plataforma/api test:s3` and `pnpm --filter @plataforma/cardano test:yaci`.

The 1 `it.fails` is intentional: it pins a known, unfixed bug in `LocationModal` (`modals-map.test.tsx`) — the map may fail to render when the modal opens with `open` set directly rather than through user interaction. It still needs to be reproduced in a real browser before deciding whether to fix the component or the test. **Update 2026-09-28:** reproduced in production (the project-detail location map opened empty) and fixed in the component; the `it.fails` became a regular `it`. The figures in the table above are those of 2026-09-21 and are left as they were.

Coverage

Package	Statements	Branches	Functions	Lines	Threshold	Source
apps/api	100% (2070/2070)	100% (838/838)	100% (424/424)	100% (1900/1900)	99/99/99/99	CI (pnpm test:coverage)
apps/web	100% (1955/1955)	100% (1854/1854)	100% (766/766)	100% (1740/1740)	100/100/100/100	CI (pnpm test:coverage)
packages/shared	100% (247/247)	100% (56/56)	100% (19/19)	100% (231/231)	100/100/100/100	CI (pnpm test:coverage)
packages/cardano	100% (295/295)	100% (149/149)	100% (90/90)	100% (273/273)	95/95/95/95	CI (pnpm test:coverage)

All four TypeScript parts measure 100% on all four metrics, all measured in CI. The SOM's criterion 2 — "unit tests $\geq 95\%$ coverage", read as covering the whole app — is met.

The contracts (contracts/) have no line coverage percentage: Aiken does not measure line coverage. The coverage evidence for the validator is [aiken-coverage-report.pdf](#): every condition under which the validator rejects a transaction has at least one test that triggers it.

Security in the same run

- **Semgrep**: 0 findings (see above).
- **Dependency scan (pnpm audit)**: 0 critical, which is the threshold that blocks the build. The full report lists 12 (4 high, 8 moderate), **all in build and test tooling**, none in what runs in production: vitest and its dependencies (undici, nanoid, postcss, @vitest/mocker) and the @tanstack/router-plugin chain (browserslist, baseline-browser-mapping). The triage is in the [security review](#).

The E2E job on 2026-09-24, flaky and non-blocking (no longer the case)

A separate job, explicitly **non-blocking** (continue-on-error: true in ci.yml). Running this same suite twice against the same commit: the first run failed 4 of 90 (86 passed), the second failed 5 of 90 (85 passed), with **different tests** failing each time (admin-certifier-invite, panels-DEV-INVESTORS-LIST, and on the second run also walkthrough-Walkthrough) — all Playwright timeouts (Test timeout of 30000ms exceeded / expect(locator).toBeVisible() failed), not a wrong business assertion. It is resource contention on GitHub Actions' shared runner (two browsers, mobile and desktop, driving the whole app), not a code defect: **locally, against the same code, the suite runs clean.**

Since 2026-10-07 the E2E suite blocks. Over the 60 runs before the change, every E2E failure was a single test, AUTH-ME-001, from a race in the test itself: it broke the token on the loaded page while goto fetched the new one, and a request from the old page cleared the session too early. Fixed in 1d2bdee (50 out of 50 locally). The suite runs in 4 shards, each with its own database, with 1 worker and no retries.

A flaky test, disclosed

One of three local runs of `pnpm test:coverage` failed in `apps/api/test/spec-213-un-bundle-por-stage.test.ts` with `SQLITE_BUSY: database is locked`. The other two passed completely, and so did the CI run. It is contention on the test's SQLite database when coverage instrumentation slows execution down, not a failure of what the test verifies.

How to reproduce

```
pnpm install
pnpm verify:all           # lint + typecheck + tests + build + Aiken
pnpm test:coverage        # coverage of the four TS parts, with their thresholds
pnpm --filter web e2e     # the E2E suite (starts web + api)
```